<u>פרויקט סוף- מערכות הפעלה</u>

מגישים: ניב קופולוביץ- 316352178

דני פורטנוב- 319397352

מוגש למרצה: עופר צור

הוגש בתאריך: 19.05.2024

<u>אופן ביצוע המטלה</u>

הדרישה במטלה היא לעבור על Database שמתאר חיבורים של חוטי מתכת המחברים טרנזיסטורים בשבב, ולחשב את ההתנגדות והקיבול של כל חוט. זאת ע"י שימוש בנושאים שנלמדו במסגרת תכנות מקבילי.

כדי לממש את המטלה בחרנו לעבוד עם Threads (ולא עם Processes- יפורט בהמשך בפירוט הבעיות בהן נתקלנו), ובנינו מספר פונקציות אותן נשייך ל- Threads השונים בהמשך:

- **ReadFromWiresFile**: פונקציה זו קוראת את המידע על כל חוט מ- Database.
- **InitMatricesAndCalResistance**: פונקציה זו מכניסה את החוטים למטריצות ומחשבת את ההתנגדות של כל חוט.
- **CalculateCapacitance**: פונקציה זו מחשבת קיבול על סמך מיקומי החוטים בשבב.
- **SaveResultsToFile**: פונקציה זו שומרת את ההתנגדות והקיבול המחושבים לקובץ תוצאות.

בנוסף לפונקציות אלה כתבנו **פונקציה ראשית (Main)** שמאתחלת את ה- Threads ומבני הנתונים הדרושים לביצוע המשימה (Queue, List).

אופן הפעולה של כל פונקציה:

1. פונקציה ראשית (Main):

- תחילה נבקש מהמשתמש להזין את מספר ה-Threads בהם ירצה להשתמש.
- לאחר מכן ניכנס ללולאה כדי לעבד כל קובץ בעל סיומת dat שנמצא בתיקייה ששמה נמצא במשתנה directoryToProcess. במקרה שלנו, אנו מעבדים את התיקייה "working_circuits_0104" ומניחים כי היא נמצאת באותה תיקייה בה התוכנית נמצאת.
- בתוך הלולאה, נאתחל תורים (fileQ, wiresQ, resultsQ) ומטריצות (verticalMatrix, hortizontalMatrix) הדרושים לעיבוד במקביל.
- כעת נפתח את הקובץ הנוכחי, נקרא את תוכנו ונחלץ את הממדים של השבב מהשורה הראשונה של הקובץ.
- ניצור Thread (readThread) שתפקידו לקרוא את נתוני החוטים מהקובץ, במקביל בזמן שפעולות אחרות נמשכות (יצירת המטריצות).
  לאחר יצירת המטריצות, ניצור Threads נוספים (כמספר ה-Threads שהוזן), כאשר כל אחד מהם מבצע במקביל את הפונקציה InitMatricesAndCalResistance (הכנסת החוטים למטריצות וחישוב ההתנגדות עבור כל חוט).
- נמתין ש- readThread יסיים לקרוא את הקובץ ולהכניס נתונים לתור fileQ. בנוסף נמתין לסיום ה- Threads שמבצעים את הפונקציה שממלאת את המטריצות עפ"י מיקום החוטים InitMatricesAndCalResistance ע"י המתנה לתור fileQ (הוא JoinableQueue, לכן ברגע שמתודת join משתחררת, הוא התרוקן וסיימנו לעבד את כל הפריטים).
- לאחר מכן ניצור Threads (כמספר ה-Threads שהוזן) חדשים שתפקידם לחשב את הקיבול של כל חוט בעזרת הפונקציה CalculateCapacitance. Threads אלה פועלים במקביל, ומחשבים קיבול עבור חוטים שונים.
- בזמן שה- Threads מחשבים את הקיבול, ניצור Thread נוסף resultsThread שתפקידו לשמור את התוצאות בקובץ עם סיומת res בעזרת הפונקציה SaveResultsToFile.
- נדפיס את הזמן בשניות שלקח לעבד כל קובץ.
- בנוסף לאחר עיבוד כל הקבצים, נדפיס את הזמן הכולל שלוקח לבצע את התוכנית.

2. ReadFromWiresFile:

- פונקציה זו עוברת על נתוני החוטים מתוך קובץ הטקסט שניתן לה שמייצג שבב ומכניסה כל נתון בנפרד לתור fileQ.

3. InitMatricesAndCalResistance:

- פונקציה זו מכניסה את נתוני החוטים למטריצות ומחשבת את ההתנגדות של כל חוט.
- הפונקציה נכנסת ללולאה אינסופית כדי לעבד את נתוני החוטים עד שfileQ מקבל אות "END".
- בכל איטרציה הפונקציה מוציאה את הנתונים של חוט מסוים מתוך התור fileQ.
- הפונקציה מחלצת את שם החוט והשדות מהפריט. נפצל את השדות כדי להפריד קואורדינטות x ו-y של מקטעי חוט.
- הפונקציה עוברת על מקטעי החוט ומחשבת את ההתנגדות של כל מקטע על סמך ההפרש בקואורדינטות x ו-y.
- בנוסף, הפונקציה מזהה מקטעים אופקיים ואנכיים ומעדכנת את המטריצות verticalMatrix, hortizontalMatrix בהתאם.
- הפונקציה מכניסה את נתוני החוט המעובדים (שם החוט, ההתנגדות והקואורדינטות) לתור wiresQ.

4. SaveResultsToFile:

- פונקציה זו שומרת את ההתנגדות והקיבול המחושבים של חוטים לקובץ.
- הפונקציה נכנסת ללולאה אינסופית עד שresultsQ מקבל אות "END".
- בכל איטרציה הפונקציה מוציאה את הנתונים המעובדים של חוט מסוים (שם החוט, התנגדות וקיבול) מתוך התור resultsQ.
- הפונקציה ממיינת את התוצאות לפי שם החוט, בהנחה ששם החוט מתחיל תמיד ב"NET" ואחריו מספר (לדוגמה NET30).
- לאחר מכן הפונקציה כותבת את התוצאות הממוינות לקובץ פלט, אותה היא בונה על ידי החלפת הסיומת ".dat" ב-".res" מהקלט filename.

5. CalculateCapacitance:

- פונקציה זו מחשבת את הקיבול של כל חוט.
- הפונקציה נכנסת ללולאה אינסופית כדי לחשב את הקיבול של כל חוט עד שwiresQ מקבל אות "END".
- בכל איטרציה הפונקציה מוציאה את הנתונים של חוט מסוים מתוך התור wiresQ.
- הפונקציה מחלצת את שם החוט, ההתנגדות והקואורדינטות.
- הפונקציה עוברת על קוארדינטות החוט, ומחשבת את הקיבול על סמך המקטעים השכנים שלו במטריצות verticalMatrix, hortizontalMatrix, על ידי סיכום ערכי הקיבול של מקטעים שכנים של כל מקטע חוט (לפי התנאים המוגדרים במטלה).
- הפונקציה מכניסה את הקיבול המחושב יחד עם שם החוט וההתנגדות לתור resultsQ.

<u>אופן ההרצה של התוכנית</u>

1. יש לוודא שקבצי Database המכילים את נתוני החוטים בשבב נמצאים בתיקייה בשם working_circuits_0104 (קבצים אלה צריכים להיות בעלי סיומת ".dat" ולהכיל מידע על חוטים בשבב ואת גודל השבב בפורמט מסוים המתואר בדרישות המטלה).
2. יש לוודא שקובץ התוכנית ותיקיית working_circuits_0104 נמצאים שניהם באותה תיקייה.
3. הפעל את main.py.
4. בחלון ה- Terminal, תינתן הנחיה להכניס (כקלט) את מספר ה-Threads בהם המערכת תשתמש.
5. בסיום עיבוד של כל קובץ יודפס הזמן שלקח לעבד את הקובץ.
6. לאחר השלמת העיבוד של כל קבצי ה- Database, התוכנית תדפיס את הזמן הכולל שלקח לעיבוד.
7. בסיום התוכנית, התוצאות המחושבות יישמרו לקבצי פלט עם סיומת ".res" באותה ספרייה. כל קובץ פלט יכיל שמות חוטים, התנגדויות וקיבולים מחושבים עבור קבצי הקלט המתאימים.

בעיות במהלך העבודה

תחילה ניסינו לממש את התוכנית בשימוש ב- Processes מספרית multiprocessing.
כשהרצנו את התוכנית בשימוש ב- Processes וב- Queue, JoinableQueue מספריית multiprocessing ראינו שאנחנו מגיעים ל-deadlock.
זה קורה כאשר משתמשים בקבצים עם הרבה שורות של מידע (cir1000.dat, cir20000.dat).

לאחר מכן הרצנו את התוכנית במוד דיבאג וראינו שכאשר משתמשים בתור JoinableQueue, התהליכים נתקעים ב-get אבל לא מבצעים task_done עבור כל get, ובעצם מדלגים על העבודה שלהם באותה פונקציה.
כתוצאה מכך ה-semaphore הפנימי של התור לא התאפס והוא הציג שיש משימות שלא בוצעו.
הדבר נכון משום שבאמת משימות לא בוצעו אבל התור התרוקן.
אותו הדבר קורה בשימוש ב-Queue הרגיל.

ניסנו לממש את התוכנית עם Queue, JoinableQueue ו-Thread מספריית threading (במקום ב-Process). ראינו שהבעיה נפתרת, התוכנית רצה עד הסוף ואנחנו לא מגיעים ל-Deadlock.

קוד התוכנית

```
#!/usr/bin/env python
from multiprocessing import JoinableQueue, Queue
from threading import Thread
import time
import os


def InitMatricesAndCalResistance(fileQ, wiresQ, verticalMatrix,
hortizontalMatrix):
    # reading wires from file so each thread reads one line at a given
time
    while True:
        xVector = []
        yVector = []
        wireResistance = 0
        item = fileQ.get()
        if item == 'END':
            return
        # fields - "(0,20),(20,20),(20,48),(48,48),(48,32)" for example
        wire_name, fields = item
        # vector - ["0,20","20,20","20,48","48,48","48,32"] for example
        vector = fields[1:-1].split('),(')
        for v in vector:
            x, y = v.split(',')
            xVector.append(int(x))
            yVector.append(int(y))

        for i in range(1, len(xVector)):
            wireResistance += abs(xVector[i] - xVector[i-1]) + \
                abs(yVector[i] - yVector[i-1])

            # horizontal line
            if yVector[i] == yVector[i-1]:
                min_x = min(xVector[i], xVector[i-1])
                max_x = max(xVector[i], xVector[i-1])
                # inserting 1's in (L x 1) segment
                for column in range(min_x, max_x):
                    hortizontalMatrix[yVector[i]][column] = 1

            # vertical line
            else:
                min_y = min(yVector[i], yVector[i-1])
                max_y = max(yVector[i], yVector[i-1])
                # inserting 1's in (1 x L) segment
                for row in range(min_y, max_y):
                    verticalMatrix[row][xVector[i]] = 1
```

```
            wiresQ.put((wire_name, wireResistance, (xVector, yVector)))
            fileQ.task_done()


def ReadFromWiresFile(file_handler, fileQ):
    for line in file_handler:
        fileQ.put(line.strip().split())
    print("Finished reading from file")


def CalculateCapacitance(wiresQ: JoinableQueue, resultsQ,
verticalMatrix, hortizontalMatrix):
    # calculating capacitance of wires with threads so each thread
works on one wire
    # at a given time
    # The x dimension of the matrix is the length of some row
    # (which is the amount of columns)
    amountOfColumns = len(verticalMatrix[0])
    # The y dimension of the matrix is the amount of rows
    amountOfRows = len(verticalMatrix)
    while True:
        wire = wiresQ.get()
        if wire == 'END':
            return
        capacitance = 0
        wireName = wire[0]
        wireResistance = wire[1]
        x = wire[2][0]
        y = wire[2][1]
        for i in range(1, len(x)):
            # check for line type - horizontal or vertical
            vertical = x[i] == x[i-1]
            minX = min(x[i], x[i-1])
            maxX = max(x[i], x[i-1])
            minY = min(y[i], y[i-1])
            maxY = max(y[i], y[i-1])
            if vertical:
                # summing capacity 4 columns to the left and right of
the vertical line
                for column in range(max(0, minX-4), min(minX+5,
amountOfColumns)):
                    # Skip the column of the current wire
                    if column == minX:
                        continue
                    for row in range(minY, maxY):
                        capacitance += verticalMatrix[row][column]
            else:
```

```
                # summing capacity 4 rows to the top and bottom of the
horizontal line
                for row in range(max(0, minY-4), min(minY+5,
amountOfRows)):
                    # Skip the row of the current wire
                    if row == minY:
                        continue
                    for column in range(minX, maxX):
                        capacitance += hortizontalMatrix[row][column]

        resultsQ.put((wireName, wireResistance, capacitance))
        # print(f"Wire {wire_name}'s capacitance calculated")


def SaveResultsToFile(fileName: str, resultsQ):
    # writing to file the resistance and capacitance of each wire
    results = []
    while True:
        item = resultsQ.get()
        if item == 'END':
            break
        results.append(item)

    print("Writing results to file...")
    # Natural sorting
    # Assuming the wire name always starts with "net"
    results.sort(key=lambda x: int(x[0][3:]))

    # The file ends with ".dat"
    outfile = fileName.replace("dat", "res")
    with open(outfile, 'w') as fw:
        for wireName, resistance, capacitance in results:
            fw.write(f"{wireName:<20} {resistance:<8}
{capacitance:<8}\n")
    print(f"{len(results)} wires calculated and exported to {outfile}")


def main():
    cpu_num = int(input("Enter the number of threads to use: "))
    directoryToProcess = os.path.join(os.getcwd(),
'working_circuits_0104')
    # Taking only files that end with ".dat"
    files = [f for f in os.listdir(directoryToProcess) if
f.endswith('.dat')]
    start = time.time()

    for filename in files:
        threads = []
        wiresQ = Queue()
```

```
        resultsQ = Queue()
        fileQ = JoinableQueue()
        filePath = os.path.join(directoryToProcess, filename)
        with open(filePath) as f:
            lines = f.readlines()
        print(f"File {filename} loaded successfully")
        _, xSize, ySize = lines.pop(0).strip().split()
        # Start to read the file before builiding the matrices, as it
takes time
        print("Reading wires from file...")
        readThread = Thread(target=ReadFromWiresFile, args=(lines,
fileQ))
        readThread.start()
        # 2 matrices with (xSize x ySize) shape for sharing between
threads
        # one is for horizontal lines and the other is for vertical
lines
        # we'll use them for calculating the capacitance of each wire
        print("Building matrices...")
        verticalMatrix = [[0 for _ in range(int(ySize))]
                          for _ in range(int(xSize))]
        hortizontalMatrix = [
            [0 for _ in range(int(ySize))] for _ in range(int(xSize))]
        print("Finished building matrices")
        threads = []
        for _ in range(cpu_num):
            t = Thread(target=InitMatricesAndCalResistance, args=(
                fileQ, wiresQ, verticalMatrix, hortizontalMatrix))
            t.start()
            threads.append(t)
        readThread.join()
        fileQ.join()
        for t in threads:  # All data was read from file, signal the
threads to end
            fileQ.put("END")
        print("Calculating capacitance...")
        threads = []
        for _ in range(cpu_num):
            t = Thread(target=CalculateCapacitance, args=(
                wiresQ, resultsQ, verticalMatrix, hortizontalMatrix))
            t.start()
            threads.append(t)
        resultsThread = Thread(target=SaveResultsToFile,
                               args=(filename, resultsQ))
        resultsThread.start()
        for t in threads:  # Signal capacitance threads to end
            wiresQ.put("END")
        for t in threads:  # Wait for threads to end
```

```
            t.join()
        resultsQ.put("END")
        print("Finished calculating capacitance")

        resultsThread.join()
        print(
            f"Finished processing file: {filename} in {(time.time() -
start)} seconds")
        print()
    print(f"Finished processing all files in {(time.time() - start)}
seconds")


if __name__ == '__main__':
    main()
```